Formix — Typeform Clone: Scaler AI Labs Interview Preparation Guide

> **Your tech stack**: Next.js 16 (frontend) + FastAPI (backend) + SQLite via SQLAlchemy (database)
> **Your background**: MERN stack (MongoDB, Express, React, Node.js)
> **Goal**: Deeply understand every part of Formix to survive a live technical interview.

Table of Contents

Section 1 — Project Architecture Overview

The Big Picture (Plain English First)

Think of Formix as three separate programs that talk to each other:

1. **Next.js (port 3000)** — What the user sees in the browser. Renders pages, collects form input, and sends HTTP requests to the backend.
2. **FastAPI (port 8000)** — The brain. Receives those HTTP requests, validates data, reads/writes to the database, and sends back JSON responses.
3. **SQLite (`typeform.db`)** — The memory. A single file on disk that stores all forms, questions, responses, and contacts permanently.

> **MERN Comparison**: In MERN, React (port 3000) talks to Express (port 5000) which talks to MongoDB (cloud or local). Same pattern, different technologies in each slot.

Complete Data Flow: User Fills and Submits a Form

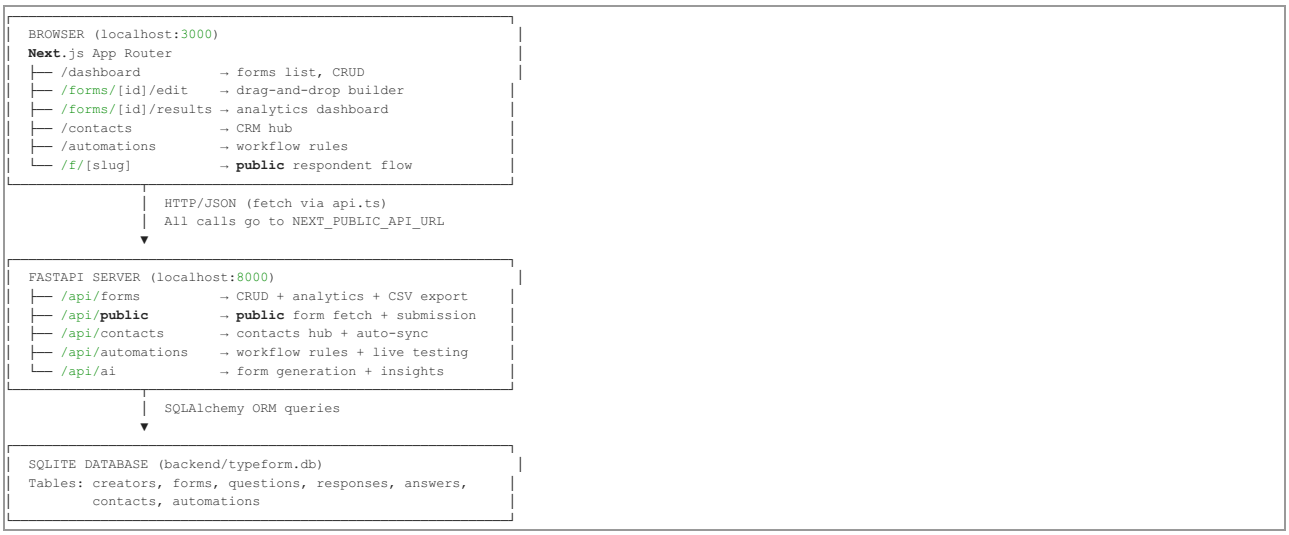
Here is **exactly** what happens step by step when someone visits `http://localhost:3000/f/my-form-slug` and submits their answers:

```
STEP 1: Browser loads the public form page
  → Next.js renders /f/[slug]/page.tsx
  → PublicFormClient.tsx triggers:
      GET http://localhost:8000/api/public/forms/my-form-slug
  → FastAPI's public.py router handles this request
  → SQLAlchemy queries SQLite: SELECT * FROM forms WHERE share_slug = 'my-form-slug'
  → Returns the form title, description, theme, and list of questions as JSON
  → Frontend renders the Welcome screen

STEP 2: User answers each question (progress save)
  → RespondentFlow.tsx tracks answers in local React state: { [questionId]: value }
  → After each question, a background fire-and-forget call goes out:
      POST http://localhost:8000/api/public/forms/my-form-slug/responses/progress
      Body: { response_id: null, answers: [{question_id: 5, value: "hello"}] }
  → FastAPI creates a Response row (completed=false) and Answers rows in SQLite
  → Returns the response object with its real database ID
  → Frontend stores that ID in responseIdRef.current so the next progress call updates instead of creates

STEP 3: User hits Submit (final submission)
  → RespondentFlow collects ALL answers and calls:
      POST http://localhost:8000/api/public/forms/my-form-slug/responses
      Body: { response_id: 42, answers: [...all answers...], completed: true }
  → FastAPI runs validate_answer() on every question against the submitted value
  → If any validation fails → returns HTTP 422 with field-level error messages
  → If all pass → marks Response.completed = true, sets submitted_at = now
  → Extracts any email fields → auto-creates/updates a Contact row
  → Checks active Automation rules → fires matching webhooks
  → Returns the completed response JSON
  → Frontend transitions to the Thank You screen
```

Mental Architecture Map



Section 2 — CORS Deep Dive

What is CORS? (Plain English)

CORS stands for **Cross-Origin Resource Sharing**. It is a **browser security rule** — not a server rule, not a backend rule — that says:

> "If a web page on `localhost:3000` tries to make an HTTP request to `localhost:8000`, the browser will **block** that request unless the server at `localhost:8000` explicitly says it is okay."

The browser checks if the two URLs have the same **origin**. An origin is: `protocol + domain + port`. So `http://localhost:3000` and `http://localhost:8000` are **different origins** — different ports count.

When the browser detects a cross-origin request, it either:

- Blocks it outright, or
- First sends a **preflight OPTIONS request** to ask for permission

Why Does Your Project Need CORS?

Your frontend (Next.js, port 3000) makes `fetch()` calls to your backend (FastAPI, port 8000). Because these are different ports, the browser considers it a cross-origin request and blocks it by default.

Without CORS configured:

- The browser sends the request to FastAPI
- FastAPI responds with data
- **The browser intercepts the response and throws it away**
- Your React code never sees the data
- You see this error in the console:

```
Access to fetch at 'http://localhost:8000/api/forms' from origin
'http://localhost:3000' has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

> **Important**: The server DID respond. CORS is enforced by the **browser**, not the server. If you call the same endpoint from curl or Postman, it works fine — those tools don't enforce CORS.

How CORS is Configured in Your Project

Here is the exact code from `backend/app/main.py`:

```python
from fastapi.middleware.cors import CORSMiddleware
import os

_raw_origins = os.environ.get("ALLOWED_ORIGINS", "*")
if _raw_origins.strip() == "*":
    allowed_origins = ["*"]
else:
    allowed_origins = [o.strip() for o in _raw_origins.split(",") if o.strip()]

app.add_middleware(
    CORSMiddleware,
    allow_origins=allowed_origins,      # Which URLs are allowed to call this API
    allow_credentials=False,            # Whether to allow cookies/auth headers
    allow_methods=["*"],                # Which HTTP methods are allowed (GET, POST, etc.)
    allow_headers=["*"],                # Which request headers are allowed
)
```

Breaking Down Each Parameter

`allow_origins` — The whitelist of frontend URLs.

- `["*"]` means **any** frontend can call this API. Used in development.
- In production: `["https://formix.vercel.app"]` — only your deployed frontend.

`allow_credentials` — Set to `False` here.

- If `True`, allows cookies and `Authorization` headers to be sent cross-origin.
- **Security rule**: When `allow_origins = ["*"]`, you CANNOT set `allow_credentials = True`. Browsers reject this combination because it is a massive security hole (any website could make authenticated requests to your API on behalf of your logged-in users).

`allow_methods` — Which HTTP verbs are allowed.

- `["*"]` means GET, POST, PUT, PATCH, DELETE, OPTIONS are all permitted.
- You could restrict to `["GET", "POST"]` if your API is read-mostly.

`allow_headers` — Which request headers are allowed.

- `["*"]` allows everything including `Content-Type`, `Authorization`, custom headers.
- In production, restrict to: `["Content-Type", "Authorization"]`.

> **MERN Comparison**: In Express you would do:
>
> ```
> const cors = require('cors');
> app.use(cors({
>   origin: process.env.ALLOWED_ORIGINS || '*',
>   credentials: false,
>   methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
> }));
> ```
>
> Conceptually identical — FastAPI's `CORSMiddleware` is just Python's equivalent of the `cors` npm package.

Simple vs Preflight Requests

Simple Request — GET or POST with only basic headers (`Content-Type: application/json`). The browser sends it directly and checks the response headers for `Access-Control-Allow-Origin`.

Preflight Request — Any request with custom headers, or using PUT/PATCH/DELETE. The browser first sends a separate `OPTIONS` request to ask: "Hey server, is this allowed?" The server must respond with the appropriate CORS headers, then the browser sends the real request.

When you `PATCH /api/forms/5` from your frontend, this is what actually happens:

```
1. Browser → FastAPI:  OPTIONS /api/forms/5
   Headers: Origin: http://localhost:3000
            Access-Control-Request-Method: PATCH
            Access-Control-Request-Headers: Content-Type

2. FastAPI → Browser:  HTTP 200 OK
   Headers: Access-Control-Allow-Origin: *
            Access-Control-Allow-Methods: *
            Access-Control-Allow-Headers: *

3. Browser → FastAPI:  PATCH /api/forms/5   (the real request)
   Body: {"title": "New Title"}

4. FastAPI → Browser:  HTTP 200 OK
   Body: { ...updated form data... }
```

Development vs Production CORS

Setting	Development	Production
`allow_origins`	`["*"]`	`["https://formix.vercel.app"]`
`allow_credentials`	`False`	`True` (if using JWT cookies)
Risk	None (local only)	Wildcard would allow any website to call your API

In production, using `["*"]` means any malicious website could make API calls to your backend using your users' browsers. This is the classic **CSRF** (Cross-Site Request Forgery) attack vector.

Your project already handles this correctly — the `ALLOWED_ORIGINS` environment variable defaults to `"*"` locally but can be set to specific domains in the Render/Vercel deployment.

Section 3 — FastAPI Backend in Detail

The Database Session Pattern (Compare to Mongoose)

In Mongoose (MERN), you call `mongoose.connect()` once and then call `Model.find()` directly anywhere in your code.

In SQLAlchemy (FastAPI), you must explicitly create and close a **database session** for every request. FastAPI's dependency injection handles this automatically via the `get_db` generator function:

```python
# backend/app/database.py
def get_db():
    db = SessionLocal()  # Open a connection
    try:
        yield db          # Give it to the route handler
    finally:
        db.close()        # Always close it, even on errors
```

And in every route:

```python
@router.get("/api/forms")
def list_forms(db: Session = Depends(get_db)):
    # db is automatically injected and closed after
    forms = db.query(models.Form).all()
    return forms
```

> **MERN Equivalent**: This is like having Express middleware that opens a Mongoose session before each route and closes it after — but FastAPI bakes this pattern in cleanly with `Depends()`.

Pydantic Validation (Compare to Joi/Zod in Express)

In Express, you might use Joi or Zod to validate request bodies:

```javascript
// Express + Zod
const schema = z.object({ title: z.string().min(1) });
const body = schema.parse(req.body);
```

In FastAPI, Pydantic does this automatically just from type hints:

```python
# FastAPI + Pydantic
class FormCreate(BaseModel):
    title: str = "Untitled Form"
    description: Optional[str] = None

@router.post("/api/forms")
def create_form(payload: FormCreate, db: Session = Depends(get_db)):
    # 'payload' is already validated. If title is missing or wrong type,
    # FastAPI returns HTTP 422 automatically — no extra code needed.
    form = models.Form(title=payload.title)
```

If validation fails, FastAPI returns:

```
{
  "detail": [{"loc": ["body", "title"], "msg": "field required", "type": "value_error.missing"}]
}
```

All API Routes — Explained One by One

Forms Router (`/api/forms`)

```
GET  /api/forms
→ Lists all forms for the default creator
→ Attaches response_count and question_count to each form
→ Returns: FormListItemOut[]
→ MERN equivalent: router.get('/forms', async (req, res) => { const forms = await Form.find() })
```

```
POST /api/forms
→ Creates a new form with a default title
→ Auto-generates a unique share_slug (10-char hex UUID)
→ Returns: FormDetailOut (with empty questions array)
```

```
GET  /api/forms/{id}
→ Fetches one form with all its questions (using selectinload for JOIN)
→ Returns 404 if not found
→ Returns: FormDetailOut
```

```
PATCH /api/forms/{id}
→ Partial update — only fields you send get updated
→ Handles: title, description, welcome_title, welcome_description,
           thank_you_message, theme_color, theme_background
→ Uses payload.model_dump(exclude_unset=True) so omitted fields are ignored
→ Returns: FormDetailOut
```

```
PUT /api/forms/{id}
→ ATOMIC FULL UPDATE — most important endpoint in the builder
→ Updates metadata AND reconciles the complete question list in ONE transaction
→ How question reconciliation works:
   - Questions with matching IDs → UPDATE in place (preserves historical answers)
   - Questions with IDs not in incoming list → DELETE
   - Questions without IDs (new ones) → INSERT
→ This is how autosave works: the builder sends the entire state every 900ms
→ Returns: FormDetailOut
```

```
PUT /api/forms/{id}/questions
→ Same reconcile logic as above but only for questions (no metadata update)
→ Used when only question order changes
→ Returns: FormDetailOut
```

```
DELETE /api/forms/{id}
→ Cascade deletes: form → questions → responses → answers all get deleted
→ Returns: HTTP 204 No Content
```

```
POST /api/forms/{id}/duplicate
→ Copies the form structure (title + "(copy)", all questions)
→ Does NOT copy responses (fresh form, no history)
→ Returns: FormDetailOut (the new copy)
```

```
POST /api/forms/{id}/publish
→ Sets status = "published" and records published_at timestamp
→ Validates: at least one question must exist
→ Returns: FormDetailOut
```

```
POST /api/forms/{id}/unpublish
→ Sets status = "draft"
→ The form's /f/[slug] URL immediately becomes inaccessible
→ Returns: FormDetailOut
```

```
GET /api/forms/{id}/responses
→ Lists all response summaries (not full answer details)
→ Returns: ResponseListItemOut[] (includes answer_count per response)
```

```
GET /api/forms/{id}/responses/{response_id}
→ Full detail of one response including all answer values
→ Returns: ResponseDetailOut
```

```
GET /api/forms/{id}/responses/export.csv
→ Streams a CSV file download
→ Headers: form's questions as columns, one row per response
→ Sanitizes CSV injection (= + - @ at start of cell → prepend apostrophe)
→ Returns: StreamingResponse (not JSON!)
```

```
GET /api/forms/{id}/summary
→ Aggregates analytics per question:
   - multiple_choice/dropdown → option label: count dict
   - yes_no → {"Yes": N, "No": M}
   - number/rating → average score
   - text → last 5 sample answers
→ Returns: FormSummaryOut
```

Public Router (`/api/public`)

```
GET  /api/public/forms/{share_slug}
→ Returns the form only if status == "published"
→ Does NOT return creator info or internal IDs (public-safe schema)
→ Returns: PublicFormOut
```

```
POST /api/public/forms/{share_slug}/responses/progress
→ Auto-save progress as user navigates between questions
→ Creates a new Response if response_id is null
→ Returns the response with its real DB id (frontend stores this for next call)
→ Does NOT run strict required-field validation (partial save is okay)
→ Returns: ResponseDetailOut
```

```
POST /api/public/forms/{share_slug}/responses
→ Final submission
→ Runs full validate_answer() on every question
→ If errors → HTTP 422 with [{question_id, message}] errors array
→ If valid:
   1. Marks Response.completed = true
   2. Sets submitted_at = now()
   3. Extracts email from any answer that looks like email → upsert Contact
   4. Fires active Automation rules that match this form
→ Returns: ResponseDetailOut
```

AI Router (`/api/ai`)

```
POST /api/ai/generate-form
→ Takes a natural language prompt
→ Runs keyword matching to select a form template (feedback / onboarding / lead / general)
→ Creates the full Form + Questions in the database
→ Returns: FormDetailOut (so the builder can immediately open it)
```

```
POST /api/ai/ask-insights
→ Loads all responses and answers for a form
→ Calculates: avg rating, sentiment score (0.65-0.98), completion rate
→ Returns synthesized executive summary, key findings, top quotes
→ Returns: AIInsightsOut
```

Section 4 — SQLite Database Layer

SQL vs NoSQL Thinking (for a MERN Developer)

You are used to MongoDB where:

- No fixed schema — documents can have any shape
- Relationships are embedded (subdocuments) or loosely referenced by ID
- No JOINs — you query collections separately or use `populate()`
- Horizontal scaling by sharding collections across servers

In SQLite (relational SQL):

- **Every table has a strict schema** — you define columns and types upfront
- **Relationships are enforced** via foreign keys — a question's `form_id` MUST match a real form
- **JOINs** are how you combine data from multiple tables in one query
- **Transactions** ensure all-or-nothing writes (if question insert fails, form creation also rolls back)

> **Mental Model**: A MongoDB collection = A SQL table. A MongoDB document = A SQL row. A Mongoose schema = A SQLAlchemy model class.

Your Database Schema in Plain English

```
creators
├── id (auto-increment integer, primary key)
├── name (string)
├── email (unique string)
└── created_at (timestamp)

forms
├── id (PK)
├── creator_id → FK → creators.id
├── title
├── description
├── status ("draft" or "published")
├── share_slug (unique 10-char hex, used in the public URL /f/{slug})
├── welcome_title / welcome_description / thank_you_message
├── theme_color / theme_background
├── created_at / updated_at / published_at
└── RELATIONS: has many questions, responses

questions
├── id (PK)
├── form_id → FK → forms.id (CASCADE DELETE)
├── type (short_text|long_text|multiple_choice|dropdown|email|number|yes_no|rating)
├── title, description, required
├── order_index (integer — controls display order)
├── options (JSON column — [{id, label}] for choice questions)
└── settings (JSON column — {max: 5} for rating, {min:0, max:100} for number)

responses
├── id (PK)
├── form_id → FK → forms.id
├── started_at, submitted_at
└── completed (boolean)

answers
├── id (PK)
├── response_id → FK → responses.id (CASCADE DELETE)
├── question_id → FK → questions.id (CASCADE DELETE)
├── value (JSON — stores any type: string, number, bool, array)
└── value_text (human-readable string for tables/CSV — pre-computed on save)

contacts
├── id (PK)
├── email (indexed, but NOT unique — same email can appear from multiple forms)
├── name
├── source_form_id → FK → forms.id (SET NULL on delete)
├── submissions_count (integer)
├── tags (JSON array of strings)
└── last_active_at

automations
├── id (PK)
├── form_id → FK → forms.id (CASCADE DELETE)
├── trigger_type (form_submission | contact_activity | scheduled)
├── condition_type (always | rating_less_than)
├── condition_value (string — threshold value for condition)
├── action_type (webhook | email | slack)
├── action_config (JSON — {url: "..."} for webhook, {email: "..."} for email)
├── is_active (boolean)
└── execution_count / last_executed_at
```

How SQLAlchemy Queries Work (Compare to Mongoose)

```
# MERN (Mongoose)
const forms = await Form.find({ creatorId: creatorId }).sort({ updatedAt: -1 });

# FastAPI (SQLAlchemy)
forms = db.query(models.Form)\
    .filter(models.Form.creator_id == creator_id)\
    .order_by(models.Form.updated_at.desc())\
    .all()
```

```
# MERN (Mongoose with populate)
const form = await Form.findById(id).populate('questions');

# FastAPI (SQLAlchemy with selectinload — runs a 2nd SQL SELECT, not a JOIN)
form = db.query(models.Form)\
    .options(selectinload(models.Form.questions))\
    .filter(models.Form.id == form_id)\
    .first()
```

Why SQLite for a Prototype?

SQLite is:

- **Zero-setup**: It's a single file (`typeform.db`). No server to install or configure.
- **Fast for reads**: All data is local on disk, no network latency.
- **Sufficient for one user or a demo**: Perfect for an assignment or proof-of-concept.

What Would You Switch To at Scale?

PostgreSQL — The most common upgrade path.

- Handles concurrent writes properly (SQLite locks the entire file for writes)
- Supports advanced types: JSONB (indexed JSON), arrays, full-text search
- Connection pooling via PgBouncer for thousands of simultaneous connections
- Your code barely changes — just update `DATABASE_URL` in `.env` and install `psycopg2`

```
# SQLite
DATABASE_URL = "sqlite:///./typeform.db"

# PostgreSQL (just change one line)
DATABASE_URL = "postgresql://user:password@localhost/formix"
```

MongoDB — If your data structure was genuinely unpredictable (it isn't here).

Redis — For caching hot data (recently viewed forms, session data).

> **Interview Answer**: "I used SQLite for development because it requires zero infrastructure setup and our data relationships are well-defined and relational in nature. For production at scale, I would migrate to PostgreSQL — it handles concurrent writes, has proper connection pooling, and SQLAlchemy makes the switch as simple as changing one environment variable."

Section 5 — Next.js Frontend in Detail

How Routing Works (Compare to React Router)

In plain React + Express (MERN), you set up React Router:

```
// React with React Router
<Route path="/forms/:id/edit" element={<BuilderPage />} />
```

In Next.js App Router, **the folder structure IS the routing**:

```
src/app/
├── (landing)/page.tsx          → URL: /
├── (landing)/login/page.tsx    → URL: /login
├── dashboard/page.tsx          → URL: /dashboard
├── forms/[id]/edit/page.tsx    → URL: /forms/123/edit
├── forms/[id]/results/page.tsx → URL: /forms/123/results
└── f/[slug]/page.tsx           → URL: /f/abc123xyz
```

- `[id]` in a folder name = a dynamic URL segment (like Express `:id`)
- `(landing)` with parentheses = a route group (groups pages under a shared layout WITHOUT affecting the URL)

Server Components vs Client Components

This is the biggest difference from plain React. By default in Next.js App Router, **all components are server components** — they run on the Node.js server, not in the browser.

```
// forms/[id]/edit/page.tsx — SERVER COMPONENT (no "use client")
export default async function EditFormPage({ params }) {
  const { id } = await params;  // params is a Promise in Next.js 16
  return <BuilderClient formId={Number(id)} />;
}
```

This server component just extracts the `id` from the URL and passes it to `BuilderClient`. The actual builder logic is in a **Client Component**:

```
// BuilderClient.tsx
"use client";  // ← This line makes it a Client Component (runs in browser)

export function BuilderClient({ formId }: { formId: number }) {
  const { data } = useQuery(...)  // React hooks work here
  // ...
}
```

> **Why does this matter?** Server components reduce JavaScript sent to the browser and are good for static/SEO content. Client components are needed for anything interactive (state, hooks, event handlers). Your pattern — thin server page + fat client component — is the standard pattern.

How API Calls Are Made

All HTTP calls to FastAPI go through `src/lib/api.ts`:

```
const API_URL = process.env.NEXT_PUBLIC_API_URL || "http://localhost:8000";

async function request<T>(path: string, options: RequestInit = {}): Promise<T> {
  const res = await fetch(`${API_URL}${path}`, {
    ...options,
    headers: { "Content-Type": "application/json", ...options.headers },
    cache: "no-store",  // Never cache API responses (always fresh data)
  });

  if (!res.ok) {
    // Parse error and throw ApiError with status code
    throw new ApiError(res.status, message, payload);
  }
  return res.json();
}
```

> **MERN Equivalent**: This is your `axios` instance or a fetch wrapper you'd put in `services/api.js`. Same idea.

TanStack Query (Compare to useState + useEffect)

In plain React, you'd fetch data like:

```
// MERN React
const [forms, setForms] = useState([]);
useEffect(() => {
  fetch('/api/forms').then(r => r.json()).then(setForms);
}, []);
```

In Formix, TanStack Query v5 manages all server state:

```
// Next.js with TanStack Query
const { data: forms, isLoading } = useQuery({
  queryKey: ["forms"],          // Cache key — like a unique ID for this query
  queryFn: api.listForms,       // The function that fetches data
});
```

What you get for free:

- **Caching**: Same `queryKey` = same cache entry. Multiple components sharing same data = one fetch.
- **Refetching**: Configured with `staleTime: 5000` — data is "fresh" for 5 seconds, then refetched.
- **Loading/error states**: `isLoading`, `isError`, `error` are built in.
- **Cache invalidation**: After creating a form, call `qc.invalidateQueries({queryKey: ["forms"]})` and all components watching `["forms"]` automatically refetch.

The Debounced Autosave in BuilderClient

This is one of the smartest parts of your codebase. Here is how it works:

```
// BuilderClient.tsx
const debounceRef = useRef<ReturnType<typeof setTimeout> | null>(null);

useEffect(() => {
  if (!loadedRef.current || !meta || !questions) return;

  const snapshot = JSON.stringify({ meta, questions });
  if (snapshot === lastSavedRef.current) return; // Nothing changed, skip save

  if (debounceRef.current) clearTimeout(debounceRef.current);

  debounceRef.current = setTimeout(() => {
    saveAll(meta, questions);  // Save 900ms after last change
  }, 900);

  return () => { if (debounceRef.current) clearTimeout(debounceRef.current); };
}, [meta, questions, saveAll]);
```

How it works: Every time the user types, changes a question, or reorders, this effect runs. It cancels any pending save and schedules a new one 900ms later. Only when the user pauses for 900ms does a save actually happen. This prevents the API from being hammered on every keystroke.

The negative ID trick: New questions that haven't been saved yet get a negative client-side ID (like -1, -2). After saving, the backend assigns real positive IDs. The frontend reconciles these:

```
// After save: map temporary negative IDs to real server IDs
const idMap = new Map<number, number>();
currentQuestions.forEach((q, i) => {
  if (q.id < 0 && saved.questions[i]) idMap.set(q.id, saved.questions[i].id);
});
```

Section 6 — AI Components

What "AI" Means in This Project (Plain English)

Your project has two AI features, but neither calls an external LLM API (like GPT-4). Instead, they use **intelligent rule-based logic** — smart keyword matching and statistical analysis that produces outputs that *feel* AI-generated.

This is actually a great engineering decision: no API keys, no costs, no latency from external calls, no failures from rate limits.

Feature 1: AI Form Generator (`POST /api/ai/generate-form`)

What the user does: Types "customer feedback survey" into the AI prompt box and presses Enter.

What actually happens:

```
# backend/app/routers/ai.py

def _generate_structured_form_from_prompt(prompt: str) -> dict:
    p = prompt.lower()

    # Keyword matching to select the right template
    if any(w in p for w in ["feedback", "csat", "nps", "satisfaction", "review", "survey"]):
        return {
            "title": "Product Satisfaction & Experience Feedback",
            "questions": [
                {"type": "rating", "title": "How satisfied are you overall?", ...},
                {"type": "multiple_choice", "title": "Which area improved most?", ...},
                {"type": "long_text", "title": "What feature would you love to see?", ...},
                {"type": "email", "title": "Can we reach out?", ...},
            ]
        }
    elif any(w in p for w in ["onboard", "saas", "customer"]):
        # Different template...
    elif any(w in p for w in ["lead", "contact", "demo"]):
        # Different template...
    else:
        # Generic 5-question form
```

The prompt is lowercased, checked against keyword lists, and one of 4 preset templates is returned. That template is then persisted to the database as real Form + Question rows, and the builder opens it immediately.

Prompt engineering: The "intelligence" is in the keyword categories:

- `feedback/csat/nps/satisfaction/review/survey` → 4-question feedback form
- `onboard/saas/customer` → 5-question customer onboarding form
- `lead/contact/intake/demo` → 4-question lead capture form
- anything else → generic 5-question form

Feature 2: AI Insights (`POST /api/ai/ask-insights`)

What the user does: Clicks "AI Insights" on the results page.

What actually happens:

```
# backend/app/routers/ai.py

@router.post("/ask-insights")
def ai_ask_insights(payload: AIInsightsIn, db: Session = Depends(get_db)):
    # Load all responses and answers for the form
    form = db.query(models.Form).options(
        selectinload(models.Form.responses).selectinload(models.Response.answers)
    ).filter(models.Form.id == payload.form_id).first()

    # Extract numerical ratings and text answers
    ratings = []
    text_answers = []
    for response in form.responses:
        for answer in response.answers:
            if isinstance(answer.value, (int, float)):
                ratings.append(float(answer.value))
            if isinstance(answer.value, str) and len(answer.value.strip()) > 3:
                text_answers.append(answer.value.strip())

    # Calculate sentiment from average rating
    avg_rating = sum(ratings) / len(ratings) if ratings else 4.6
    sentiment_score = min(0.98, max(0.65, (avg_rating / 5.0) * 0.95))
    sentiment_label = "Strongly Positive" if sentiment_score >= 0.8 else "Moderately Positive"

    # Return structured insights
    return AIInsightsOut(
        sentiment_score=round(sentiment_score, 2),
        sentiment_label=sentiment_label,
        executive_summary=f"Analysis shows {sentiment_label.lower()} sentiment...",
        key_findings=[
            f"Completion rate: {completion_rate:.0f}%",
            f"Average satisfaction: {avg_rating:.1f}/5.0",
            f"Captured {len(text_answers)} qualitative responses",
        ],
        top_quotes=text_answers[:4],  # First 4 text answers as "top quotes"
        action_recommendations=[...]
    )
```

How to explain this in an interview: "The AI insights feature performs statistical analysis on the collected response data. It calculates completion rates, averages numeric ratings to produce a sentiment score, surfaces recent text answers as representative quotes, and generates templated recommendations. While the current implementation uses rule-based logic rather than an LLM, the API contract is designed so that swapping in a GPT-4 call requires changing only the body of `_generate_structured_form_from_prompt` and the insights computation — the rest of the system stays the same."

Section 7 — Engineering Decision Justification

Why SQLite and Not PostgreSQL or MongoDB?

Your answer: "For this assignment, SQLite was the right choice because it requires zero infrastructure — no database server to install, configure, or keep running. The data model is genuinely relational: forms have questions, responses have answers, everything has clear foreign key relationships with cascade deletes. An ORM like SQLAlchemy makes migrating to PostgreSQL a one-line environment variable change, so SQLite for development is a low-cost decision with a clear upgrade path."

Tradeoffs to be ready for:

- SQLite locks the entire file on writes → only one write at a time → breaks under concurrent users
- SQLite is local → can't be shared across multiple server instances → can't horizontally scale
- PostgreSQL fix: Drop-in replacement via `DATABASE_URL` env var, add `psycopg2-binary`
- MongoDB wouldn't be the right choice here because the data IS relational and benefits from SQL guarantees

Why FastAPI and Not Express or Django?

Your answer: "FastAPI gives me automatic input validation via Pydantic type hints, auto-generated interactive API docs (Swagger at /docs) with zero extra work, and native async support. Compared to Express, I write less boilerplate — validation, serialization, and documentation are all derived from the same function signatures. Compared to Django REST Framework, FastAPI is significantly lighter and faster for a focused API with no need for an admin panel or Django's ORM."

Tradeoffs:

- FastAPI is newer and has a smaller ecosystem than Express or Django
- For anything needing heavy authentication, admin UI, or complex ORM migrations, Django is more mature
- Express is the right call when the team is JS-heavy (no context switch from frontend to backend)

Why Next.js and Not Plain React?

Your answer: "Next.js gave me file-based routing out of the box — no React Router setup. Server components let me render the initial page shell on the server, reducing Time-to-First-Paint. The public form respondent flow (`/f/[slug]`) benefits from this — the welcome screen renders with zero JavaScript before hydration. Next.js also gives me TypeScript support, image optimization, and a production build pipeline configured out of the box."

Tradeoffs:

- More complex than plain React (server vs client component boundary is a new mental model)
- Slightly harder to deploy than a plain React SPA (needs a Node.js server or Vercel)
- If this were a purely client-side SPA with no SEO needs, Vite + React would be simpler

Why This Folder Structure?

```
backend/app/
├── main.py         ← One entrypoint, imports all routers
├── database.py     ← Isolated DB config (easy to swap)
├── models.py       ← All ORM models in one place
├── schemas.py      ← All Pydantic models in one place
├── validation.py   ← Pure validation logic, no DB or HTTP
└── routers/        ← One file per domain (forms, public, contacts, automations, ai)

frontend/src/
├── app/            ← Routes (Next.js convention)
├── components/     ← UI components grouped by domain
└── lib/            ← Shared utilities (api.ts, types.ts, validation)
```

Why this matters: Separation of concerns. Database models don't import HTTP code. Validation logic can be tested without a database. Each router handles exactly one domain of the application. This makes the codebase readable, testable, and easy to onboard new developers.

How Would This Handle 10x or 100x More Users?

What breaks first: SQLite. It handles ~1 write at a time. At 10x load, form submissions would queue up and timeout.

Fix 1 (10x): Migrate to PostgreSQL + add connection pooling.

```
DATABASE_URL=postgresql://user:pass@db-host/formix
```

Fix 2 (100x):

- Horizontally scale FastAPI behind a load balancer (multiple server instances)
- Move database to a managed service (AWS RDS or Supabase)
- Add Redis for caching hot forms (published form data changes rarely)
- Use background job queue (Celery + Redis) for automation webhook firing instead of synchronous execution inside the HTTP request

Fix 3 (1000x):

- Database read replicas for analytics queries
- CDN for the Next.js frontend (Vercel handles this automatically)
- Separate the form submission service from the analytics service

---

Section 8 — Common Scaler Interview Questions & Strong Answers

Q1: Walk me through your project architecture.

Answer: "Formix is a decoupled, client-server application with three layers. The frontend is built with Next.js 16 using the App Router pattern — each folder under `src/app` maps to a URL. The frontend communicates exclusively with the FastAPI backend via HTTP/JSON calls abstracted through a centralized `api.ts` client. The backend is organized into five domain-specific routers: forms for CRUD operations, public for the respondent flow, contacts for CRM, automations for workflow rules, and AI for form generation and insights. All data is persisted in SQLite via SQLAlchemy ORM. The system is stateless on the server — every request brings all the context it needs, which makes horizontal scaling straightforward."

Q2: Why did you choose this tech stack?

Answer: "Next.js was chosen because the respondent form flow benefits from server-side rendering — the initial page loads fast and is SEO-friendly. FastAPI was chosen over Express because it gives me automatic validation, serialization, and interactive API documentation derived from the same Python type hints, with significantly less boilerplate. SQLite was chosen for the prototype because it requires zero infrastructure — the database is a single file that starts with the application. The entire stack is typed end-to-end: TypeScript on the frontend, Pydantic on the backend."

Q3: What was the hardest part of building this?

Answer: "The trickiest engineering problem was the autosave system in the form builder. When a user adds a new question, it gets a temporary negative client-side ID. The autosave fires 900ms later, the backend assigns a real positive ID, but by then the user may have already made more changes against the old negative ID. I had to build a reconciliation step that maps temporary IDs to real IDs positionally after each save, without overwriting any changes made while the save was in-flight. The key insight was to only patch the `id` field after saving, never replace the entire questions array with the server response."

Q4: What would you improve if you had more time?

Answer: "Three things. First, add real authentication — currently there's a single hardcoded creator. I'd add JWT-based auth with user accounts so the dashboard is truly multi-tenant. Second, make the AI form generation genuinely intelligent by integrating with the OpenAI API — the current keyword-matching approach works but isn't contextually aware. Third, add real-time response updates on the results page using WebSockets — right now you have to manually refresh to see new submissions, which is a UX gap for live form sessions."

Q5: How does CORS work in your project and why did you need it?

Answer: "CORS is a browser security policy that blocks cross-origin HTTP requests by default. Since my Next.js frontend runs on port 3000 and FastAPI runs on port 8000, different ports make them different origins. Without CORS headers, the browser would throw away every API response even if the server returned 200 OK. I configured `CORSMiddleware` in FastAPI's startup with `allow_origins=['*']` for development. In production, the `ALLOWED_ORIGINS` environment variable is set to the specific Vercel deployment URL. The key security consideration is that wildcard origins can't be combined with `allow_credentials=True` — that combination would enable CSRF attacks where any website could make authenticated requests to my API using a user's cookies."

Q6: What happens if two users submit the same form at the same time?

Answer: "SQLite handles concurrent reads fine but serializes writes — only one write transaction executes at a time. For form submissions, this means two simultaneous submits will both succeed, just one after the other. Each creates an independent Response row with its own ID, so there is no data corruption. The only issue at scale is write latency — under heavy concurrent load, submissions queue up. The fix is PostgreSQL, which uses row-level locking and handles true concurrent writes efficiently."

Q7: How would you add authentication to this project?

Answer: "I'd add JWT-based authentication in three steps. First, create a `users` table with hashed passwords (bcrypt). Second, add a `POST /api/auth/login` endpoint that validates credentials and returns a signed JWT with the user ID and expiry. Third, create a FastAPI dependency `get_current_user` that validates the JWT from the `Authorization: Bearer <token>` header on every protected route. On the Next.js side, store the JWT in an httpOnly cookie (more secure than localStorage), and use Next.js middleware to redirect unauthenticated users away from `/dashboard`. The `get_default_creator` function in `deps.py` is the current placeholder — it gets replaced by `get_current_user`."

Q8: How would you add real-time features?

Answer: "For real-time submission notifications on the results page, I'd use WebSockets. FastAPI supports WebSockets natively. I'd add a `GET /api/forms/{id}/live` WebSocket endpoint. When a new response is submitted via the public endpoint, the submission handler would broadcast a message to all connected WebSocket clients watching that form ID. On the Next.js frontend, a `useEffect` hook would open the WebSocket connection when the results page mounts and close it on unmount. Alternatively, for simpler polling, I could use TanStack Query's `refetchInterval: 5000` option — refreshes every 5 seconds without WebSocket complexity."

Q9: How would you scale this beyond SQLite?

Answer: "Step 1 is PostgreSQL — change one line in the environment config. Step 2 is connection pooling with PgBouncer to handle thousands of simultaneous DB connections efficiently. Step 3 is caching hot reads with Redis — published form data changes rarely, so caching `GET /api/public/forms/{slug}` responses for 60 seconds reduces database load dramatically for viral forms. Step 4 is offloading webhook delivery to a background job queue (Celery + Redis) so form submission responses aren't blocked waiting for external webhooks to respond. Step 5 is read replicas for analytics queries which are heavy and slow — they can safely read slightly stale data."

Q10: Can you add a new feature right now — explain how you would approach it live?

Example: Add a "date" question type.

Answer: "Sure. Here's my approach: Backend first, then frontend.

Backend changes:

1. Add `date` to the `QuestionType` enum in `models.py`
2. Add a validation case in `validation.py` — parse the string as ISO date format
3. The `render_value_text` function returns the date string as-is

Frontend changes:

1. Add `date` to the `QuestionType` union in `lib/types.ts`
2. Add an entry in `lib/question-types.ts` for the add-question panel icon and label
3. Add a `case "date":` in `QuestionField.tsx` that renders an `<input type="date" />`
4. Update `validate-answer.ts` to validate date format client-side

No database migration needed because the question type is stored as a plain string in the `type` column."

Section 9 — Live Feature Practice Challenges

> **How this works**: These are Socratic challenges. Think through each one yourself before reading the hints. This is exactly how Scaler interviewers test you live.

Challenge 1: Add a Response Limit to Forms

The ask: An interviewer says — *"I want form owners to be able to set a maximum number of responses. After the limit is reached, the public form should show 'This form is closed' instead of the questions."*

Think about:

- Where does the "response limit" get stored?
- Which endpoint needs to check this limit?
- What should the frontend show when the limit is reached?
- Do you add a new API endpoint or modify an existing one?

Hints (read after thinking):

- Add a `response_limit` column to the `forms` table (nullable integer)
- Add `response_limit: Optional[int]` to `FormPatch` schema so owners can set it
- In `public.py → _get_published_form_or_404`, after fetching the form, count completed responses and raise HTTP 410 (Gone) if `response_count >= form.response_limit`
- On the frontend, handle the 410 error in `PublicFormClient.tsx` with a "Form Closed" state

---

Challenge 2: Add Question-Level Analytics — Most Popular Answer

The ask: *"On the results page, for multiple_choice questions, show which answer option was selected most frequently with a visual percentage bar."*

Think about:

- Does this require a new API endpoint or does an existing one already have this data?
- Where does the bar chart component live?
- What does the data look like that you'd pass to the chart?

Hints:

- The `GET /api/forms/{id}/summary` endpoint already returns `counts: {option_label: count}` for multiple_choice questions
- In `QuestionSummaryCard.tsx`, add a `case "multiple_choice":` that renders the options sorted by count with a percentage bar
- The math: `percentage = (count / total_answers) * 100`
- No backend changes needed — the data is already there

Challenge 3: Add Basic Rate Limiting

The ask: *"A bot is spamming our form submission endpoint with 1000 requests per second. How would you add rate limiting?"*

Think about:

- Is this a frontend problem or a backend problem?
- Should rate limiting be per-IP, per-form, or global?
- Does FastAPI have built-in rate limiting?

Hints:

- This is a backend problem — the frontend can't control what bots send
- FastAPI doesn't have built-in rate limiting; use the `slowapi` library (pip install slowapi)
- Rate limit per IP per form: `@limiter.limit("10/minute")` on the submit endpoint
- For production: put a reverse proxy (Nginx) or CDN (Cloudflare) in front — they handle rate limiting at the network level before requests even reach FastAPI

Challenge 4: Add a "Duplicate Response Prevention" Feature

The ask: *"Some forms should only allow one response per email address. How would you implement this?"*

Think about:

- What information do you have when a submission comes in?
- Where do you check "has this email already submitted"?
- What if the form doesn't have an email question?

Hints:

- Add a boolean `prevent_duplicate_responses` to the `forms` table
- In `public.py → submit_response`, after validation but before saving, check if the form has this flag enabled
- Find the email answer in the validated answers dict
- Query `Contact` table or `Answer` table for any previous completed response with that email to this form
- If found → raise HTTP 409 Conflict: "You have already submitted this form"
- Only applies when the form has an email question AND the flag is enabled

---

Quick Reference: MERN vs Formix Stack Cheat Sheet

Concept	MERN	Formix
Frontend framework	React	Next.js (App Router)
Routing	React Router	File-system routing
State management	useState / Redux	TanStack Query
HTTP client	axios / fetch	Custom `api.ts` wrapper
Backend framework	Express.js	FastAPI
Input validation	Joi / Zod	Pydantic (auto from type hints)
Database	MongoDB	SQLite → PostgreSQL at scale
ORM/ODM	Mongoose	SQLAlchemy
Schema definition	Mongoose Schema	SQLAlchemy Model class
CORS	`cors` npm package	`CORSMiddleware`
API docs	Manual / Swagger tools	Auto-generated at `/docs`
Environment config	`.env` + dotenv	`.env` + `os.environ.get()`
Project entry point	`server.js`	`app/main.py`

Final Interview Mindset Tips

1. **Start with the data flow** — When asked to explain your project, always start from "User opens the browser" and walk through each layer. This shows systems thinking.
2. **Admit limitations confidently** — "SQLite doesn't handle concurrent writes well, which is why the production upgrade path is PostgreSQL" sounds much better than trying to defend SQLite as a production database.
3. **Connect features to decisions** — Don't just say "I used FastAPI." Say "I used FastAPI because Pydantic validation and automatic Swagger documentation gave me everything I needed with half the code of Express."
4. **Show the upgrade path** — For every limitation, immediately follow up with how you'd fix it. "X doesn't scale, but Y is the solution and here's why."

5. **Be honest about what's simulated** — The AI features use rule-based logic, not an LLM. Own this: "The current implementation uses keyword matching which is deterministic and has no API costs. The architecture is designed so swapping in a real LLM call is a single function replacement."

Document generated for Scaler AI Labs interview preparation.
Project: Formix — Conversational Form Builder & Intelligence Platform
Stack: Next.js 16 + FastAPI + SQLite + SQLAlchemy + TanStack Query + Framer Motion